

Chapter 17: Model Context Protocol (MCP) and Multi-Agent Swarm Orchestration

Preceding Bridge: In [Chapter 07: Multi-Agent Collaboration](#) and [Chapter 16: Hierarchical Episodic Memory](#), you learned how individual agents think, use tools, and retain memory. In this chapter, we explore how modern production agents interface with external enterprise infrastructure via Anthropic's **Model Context Protocol (MCP)**, and how multi-agent swarms reach deterministic consensus without infinite ping-pong loops.

1. Plain-English Jargon Demystifier

Technical Term	Plain English Translation	Real-World Metaphor
MCP (Model Context Protocol)	An open JSON-RPC 2.0 standard allowing any AI model to securely connect to external tools and data sources.	The USB-C port for AI: one universal cable connects your laptop to monitors, drives, and chargers.
MCP Host	The application running the AI assistant (e.g., Claude Desktop, Antigravity IDE, Cursor).	The laptop computer that has the USB-C port.
MCP Client	The software connector inside the host that negotiates with external tool servers.	The USB host controller on the motherboard.
MCP Server	A standalone microservice exposing specific tools, resources, or prompts via JSON-RPC.	An external hard drive, camera, or keyboard plugged into the USB port.
Multi-Agent Swarm	A network of specialized agents cooperating to solve complex, multi-stage goals.	A surgical team: lead surgeon, anesthesiologist, scrub nurse, and monitoring technician.
Blackboard Architecture	A shared global memory space where agents post partial findings and inspect others' outputs.	A physical whiteboard in a war room where detectives pin clues and connect string.
Handoff Token	A structured state object passed from one agent to another during delegation.	A relay race baton passed from Runner 1 to Runner 2.

2. Spoon-Fed Mental Model: The USB-C Standard for AI Agents

Before 2024, every AI framework had proprietary, non-interoperable tool definitions:

- LangChain had its custom `@tool` decorator.
- LlamaIndex had its custom `FunctionTool`.
- AutoGen had its custom function registry.
- OpenAI and Anthropic had different JSON schema payload formats.

If an enterprise built a secure PostgreSQL tool, they had to rewrite the tool 5 times for 5 different frameworks!

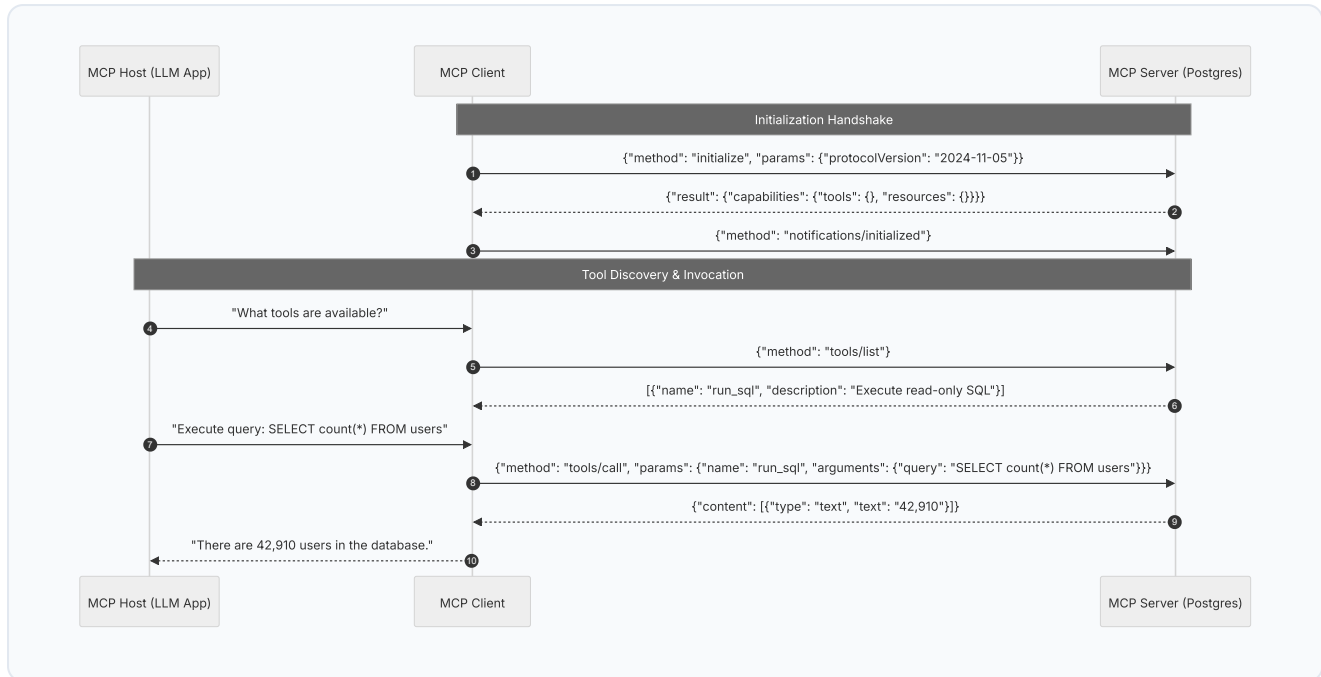
Anthropic's Model Context Protocol (MCP) unified the entire ecosystem into a standard client-server protocol over JSON-RPC 2.0:

Now, write a tool **once** as an MCP server in Python, TypeScript, or Go, and **every** AI host in the world can instantly discover its tools, call them, and inspect resources securely.

3. The 3 Primitives of MCP

Every MCP server exposes three core capabilities:

1. **Tools:** Executable functions that take JSON parameters and perform actions (e.g., `execute_sql_query`, `deploy_kubernetes_pod`).
2. **Resources:** Read-only data URIs representing file contents, database schemas, or system logs (e.g., `postgres://prod_db/schema`, `file:///logs/app.log`).
3. **Prompts:** Pre-engineered templates with instructions and context that guide the model for specific workflows (e.g., `review_pr_diff`, `trriage_incident`).



4. Multi-Agent Swarm Topologies: Avoiding the Infinite Loop

When multiple agents interact, how do you prevent them from getting stuck in an endless loop (*Agent A asks Agent B, Agent B asks Agent A back*)?

5. Junior vs Staff Implementation: Agent Tool Integration

6. Complete Runnable Implementation: Pure-Python MCP Server & Swarm Orchestrator

Here is a 100% runnable, zero-external-dependency script implementing an MCP-compatible JSON-RPC tool server and a deterministic 2-agent consensus swarm:

7. Chapter Milestone Check

Verify your understanding before continuing:

1. Why is the Model Context Protocol (MCP) considered the "USB-C" of the AI industry?

- *Answer:* It provides a single universal, standardized communication protocol (JSON-RPC 2.0) for exposing tools, prompts, and resources. Developers implement tools once on an MCP server, and any AI application can immediately connect and consume them without custom SDK integration.

2. What are the three core primitives exposed by an MCP server?

- *Answer:* Tools (callable actions), Resources (read-only data URIs), and Prompts (reusable LLM interaction templates).

3. How does a supervisor multi-agent architecture prevent infinite recursion between agents?

- *Answer:* Worker agents are not permitted to delegate directly to each other; all communication routes back through the supervisor, which tracks a strict finite turn budget and halts the workflow when convergence criteria are met.
-

Verified Worked Example and Version Note (MCP SDK)

A real MCP server with one tool and one resource is in `examples/ex04_mcp_server.py`, verified on `mcp=2.3.0`. **The Python SDK changed between major versions:** in 1.x the server class is `mcp.server.fastmcp.FastMCP`; in 2.x it is `mcp.server.mcpserver.MCPServer`. Type hints and the docstring become the tool's JSON Schema in both. Always pin the SDK version and check the migration notes when upgrading.

Pinned for the verified examples: `langgraph=1.2.13`, `mcp=2.3.0`, `pytest=9.1.1` (see `examples/requirements.txt`). All examples run offline with a scripted fake model: `cd examples && pip install -r requirements.txt && pytest -q`.

Further Reading

- [Model Context Protocol](#)
 - [MCP Python SDK](#)
 - [OpenAI Swarm](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. What problem does MCP solve?

It standardises how AI applications connect to tools and data, turning M-by-N integrations into M plus N.

▼ 2. Name MCP's three primitives.

Tools, resources and prompts.

▼ 3. What changed in the Python SDK between 1.x and 2.x?

The server class `FastMCP` (`mcp.server.fastmcp`) became `MCPServer` (`mcp.server.mcpserver`); pin the version.

▼ 4. Why treat third-party MCP servers as untrusted?

Their tool descriptions enter the prompt and their code runs on your behalf: tool poisoning and injection are possible.